SPL-1 Learning Pathway: Revised Task 1 and Task 2

Task 1: verify the sensor ID mapping. Task 2: wide-format CSV parser (one row = one timestamp, all 207 speeds). Rule: you do not move to the next step until you can explain the current one without looking.

0. Read this first

What changed from the previous PDF. The struct now holds all 207 speeds per row, so no trimming and no 6-sensor selection happen in these two tasks. The long-format loader and the trimming steps in the previous PDF are superseded. The mini-exercises in that PDF are still useful; condensed versions are repeated here.

Honest assessment of the revised plan. It is a simpler first commit and a good one. It also has four weak spots you should know about now:

1. **The struct has no sensor IDs.** A speed's position in the vector (0 to 206) is the only link to its sensor. If you ever mix up position and ID, the program will not crash; it will silently use the wrong sensor. That is exactly why Task 1 matters more than its 5-minute size suggests.
2. **Deferring the 6-sensor choice is fine; dropping it is not.** Before the Dijkstra section you still need a set of sensors that are connected in the distances file, with at least one pair that has two possible routes.
3. **File size.** The full CSV is large. A mock file with the same shape (34,272 rows x 207 columns) was 68 MB for me; yours may differ, check with ls -lh. GitHub warns above 50 MB and rejects files above 100 MB. Do not commit the full CSV. Keep it out of git (section 2.5). A big file committed once stays in the history even if you delete it later.
4. **Memory and speed.** About 7.1 million doubles is roughly 57 MB of RAM for the speeds alone. That is fine on a laptop. On my test the loader took about 1.7 seconds without optimisation. Do not optimise anything tonight.

What I could not verify. I have not seen your files. Whether the speed CSV has a header, whether that header holds sensor IDs, and what graph_sensor_ids.txt looks like inside are all things Task 1 makes you check. The loader below assumes line 1 of the speed CSV is a header and skips it; Task 1 tells you whether that is true.

Overall success criteria

#	Criterion	How you check it
1	You can state in one sentence how column j of the speed file maps to a sensor ID and to the IDs in distances_la_2012.csv.	Write the sentence in docs/ or on paper; paste the first 5 lines of each file to me if unsure.
2	The program prints the row count, then 5 rows with timestamp and first 3 speeds.	Compare against the first lines of the CSV.
3	Row count = number of lines in the CSV minus 1 (header). Every row has 207 speeds.	wc -l on the file; the program prints the first row's speed count.
4	The full CSV is not in the commit.	git status before commit; git show --stat after.
5	You can answer every self-check question aloud.	Sections 1.3 and 2.6.

Suggested time (estimates, not measured)

- Task 1: 5 to 15 minutes.
- Task 2: about 1.5 to 2.5 hours including the exercises.

1. Task 1: Verify the ID mapping

1.1 What you must understand first

You are working with three related things (as I understand the DCRNN data release; confirm in the repository README):

Item	What it is	Identifies sensors by
Speed file (METR-LA CSV)	Speeds over time. One row per timestamp, one column per sensor.	Column position, and possibly an ID in the header
graph_sensor_ids.txt	The list of the 207 sensor IDs, in the order the speed columns use.	ID (a number like 717447), by position in the list
distances_la_2012.csv	Graph edges: from, to, cost (road distance between two sensors).	ID, not position

Position vs ID. Position is where something sits in a list (0, 1, 2 ... 206). ID is the sensor's own label (for example 717447). The speed vector uses position. The distances file uses ID. To connect them you need the list in graph_sensor_ids.txt: the sensor at position j has ID = list[j]. This translation is the whole point of Task 1.

```
position j:        0        1        2     ...    206
graph_sensor_ids:  773869   767541   767542 ...   (207th ID)
speeds[j]:         speed of 773869 | speed of 767541 | ...   (same order)
distances file:    from=767541, to=767542, cost=...   (uses IDs)
```

Why you check instead of assume. Some copies of this dataset use internal indices instead of IDs, or reorder columns. If the files disagree, every later result is wrong without any error message.

1.2 Step-by-step

Step 1. graph_sensor_ids.txt. Open it in a plain text editor. Write down: (a) is it one line with commas, or one ID per line; (b) how many IDs (expect 207); (c) the first 5 and last 5 IDs.

Step 2. distances_la_2012.csv. Open it in a text editor. Write down the header and the first 5 rows. Are from and to numbers like the IDs in Step 1?

Step 3. Cross-check. Take 3 different from values and 3 different to values from the distances file. Search for each in graph_sensor_ids.txt. All 6 must be found. If any is missing, the files use different ID schemes: stop and paste the lines to me.

Step 4. Speed file header. Open the speed CSV in a text editor (it is big; a good editor will open it, or use head -c 2000). Look only at line 1. Write down whether it is a header, and if so whether it contains IDs. If it does, compare its first 5 and last 5 IDs with Step 1, in order. They should match. If line 1 has no IDs (or is not a header at all), then graph_sensor_ids.txt order is the *only* mapping; note that.

Step 5. Write the result. One short note, for example docs/data_notes.md, with the facts from Steps 1 to 4 and your one-sentence mapping rule. Committing this note is optional but cheap and useful.

Command-line alternatives if you prefer a terminal:

```
head -n 5 data/raw/distances_la_2012.csv
head -c 300 data/raw/graph_sensor_ids.txt
head -c 300 data/raw/metr-la.csv
wc -l data/raw/metr-la.csv
```

Adjust the paths to where your files actually are.

1.3 Self-check: explain aloud

1. What is the difference between a sensor's position and its ID? Which one does the speeds vector use?
2. Which file lets you translate position to ID?

3. Why does a mismatch here produce wrong results instead of an error?
4. Does your speed file's header contain IDs? If yes, do they match graph_sensor_ids.txt in order?

1.4 Task 1 success criteria

- All 6 sampled IDs from the distances file are found in graph_sensor_ids.txt.
- You have written down the header facts of all three files.
- You can state the mapping rule in one sentence.

If you want me to confirm it directly, paste the first 5 lines of the distances file, the beginning of graph_sensor_ids.txt, and the first line of the speed CSV (first 300 characters is enough).

2. Task 2: Build the wide-format CSV parser

2.1 Concepts, in the order you need them

Concept	What to understand	Exercise
Header and source files	Declaration (what exists) goes in the .h; definition (how it works) in the .cpp. #pragma once stops the header being included twice in one build.	3
struct holding a vector	TrafficRecord now contains a std::string and a std::vector<double>. Each record owns its own vector of 207 numbers.	2
std::vector	Resizable array. push_back adds at the end. size() returns the count. speeds[0] is the first element (indexing starts at 0).	2
File input	std::ifstream opens a file; is_open() tells you if it worked; std::getline(file, line) reads one line.	1
Splitting text	std::stringstream plus std::getline(ss, field, ',') reads up to each comma.	1
Text to number	std::stod converts text to double and throws an exception if it cannot.	1
size_t	The unsigned integer type used for sizes and indexes. Use it for loop counters over vectors.	2
Compiling several files	Pass every .cpp to g++. Relative paths in the program are relative to where you run it.	3

2.2 Mini-exercises (throwaway files outside the repo, not committed)

If you already did Exercises A to E from the previous PDF you may skip 1 and 3. Compile with: g++ -std=c++17 -Wall -Wextra file.cpp -o out. Type the code yourself.

Exercise 1. Split a line and convert to numbers. Success: prints three doubles; then change one value to abc and explain the crash.

```
#include <iostream>
#include <sstream>
#include <string>
#include <vector>

int main() {
    std::string line = "2012-03-01 00:00:00,64.375,67.625,0.0";
    std::stringstream ss(line);
    std::string field;
    std::vector<std::string> fields;
    while (std::getline(ss, field, ',')) fields.push_back(field);

    std::vector<double> speeds;
    for (size_t i = 1; i < fields.size(); ++i)
        speeds.push_back(std::stod(fields[i]));
    std::cout << fields[0] << " has " << speeds.size() << " speeds\n";
}
```

Exercise 2. Struct with a vector, stored in a vector. Success: prints 2, 3 and 61.

```
#include <iostream>
#include <string>
#include <vector>

struct TrafficRecord {
    std::string timestamp;
    std::vector<double> speeds;
};

int main() {
    std::vector<TrafficRecord> rows;
```

```
    TrafficRecord a;
    a.timestamp = "t1";
    a.speeds.push_back(50);
    a.speeds.push_back(60);
    a.speeds.push_back(61);
    rows.push_back(a);
    rows.push_back(a);
    std::cout << rows.size() << " " << rows[0].speeds.size()
              << " " << rows[1].speeds[2] << "\n";
}
```

Exercise 3. Two-file program. Put int add(int a, int b); in math.h (with #pragma once), the definition in math.cpp, call it from main.cpp. Compile with g++ main.cpp math.cpp -o out. Then leave math.cpp out of the command and read the linker error. Success: you can explain it.

2.3 Build the real thing, step by step

Type the files yourself. The code below compiled without warnings under -Wall -Wextra and ran correctly on a mock file with the same shape (34,272 rows x 207 sensor columns, Windows line endings); you must still run it on your real file.

Step 1. src/DataLoader.h:

```
#pragma once
#include <string>
#include <vector>

struct TrafficRecord {
    std::string timestamp;
    std::vector<double> speeds;
};

std::vector<TrafficRecord> loadCSV(const std::string& path);
```

Step 2. src/DataLoader.cpp, part 1: helpers. The anonymous namespace keeps them private to this file.

```
#include "DataLoader.h"

#include <fstream>
#include <iostream>
#include <sstream>

namespace {

// "a,b,c" -> {"a","b","c"}
std::vector<std::string> splitByComma(const std::string& line) {
    std::vector<std::string> fields;
    std::stringstream ss(line);
    std::string field;
    while (std::getline(ss, field, ',')) {
        fields.push_back(field);
    }
    return fields;
}

// Windows files end lines with "\r\n"; getline leaves the '\r' behind
void stripCR(std::string& line) {
    if (!line.empty() && line.back() == '\r') {
        line.pop_back();
    }
}

}  // namespace
```

Step 3. src/DataLoader.cpp, part 2: loadCSV. Three blocks: open, skip header, one line becomes one record.

```
std::vector<TrafficRecord> loadCSV(const std::string& path) {
```

```
    std::vector<TrafficRecord> records;

    // Block 1: open
    std::ifstream file(path);
    if (!file.is_open()) {
        std::cerr << "Error: cannot open " << path << "\n";
        return records;
    }

    // Block 2: skip the header line (assumes line 1 is a header)
    std::string line;
    if (!std::getline(file, line)) {
        return records;
    }

    // Block 3: one line = one TrafficRecord
    while (std::getline(file, line)) {
        stripCR(line);
        if (line.empty()) continue;

        std::vector<std::string> fields = splitByComma(line);

        TrafficRecord r;
        r.timestamp = fields[0];
        for (size_t i = 1; i < fields.size(); ++i) {
            r.speeds.push_back(std::stod(fields[i]));
        }
        records.push_back(r);
    }
    return records;
}
```

If Task 1 showed that line 1 of your file is *data*, not a header, delete the Block 2 read; you will then see one extra row in the count. Note: the first cell of a header may be empty. We do not parse the header tonight, so that does not matter.

Step 4. src/main.cpp:

```
#include <iostream>

#include "DataLoader.h"

int main() {
    std::vector<TrafficRecord> records = loadCSV("data/traffic_data.csv");
    std::cout << "Loaded " << records.size() << " rows\n";
    if (records.empty()) return 1;

    std::cout << "Speeds in first row: " << records[0].speeds.size() << "\n";

    for (size_t i = 0; i < 5 && i < records.size(); ++i) {
        std::cout << records[i].timestamp << " |";
        for (size_t j = 0; j < 3 && j < records[i].speeds.size(); ++j) {
            std::cout << " " << records[i].speeds[j];
        }
        std::cout << "\n";
    }
    return 0;
}
```

The first two output lines (row count and speeds per row) are extra checks beyond your spec; they are cheap and they catch most mistakes. Change the path in main.cpp to wherever your full CSV really is (for example data/raw/metr-la.csv), or copy it to data/traffic_data.csv and keep it ignored by git.

Step 5. Compile and run from the repo root (the folder containing data/ and src/):

```
g++ -std=c++17 -Wall -Wextra src/main.cpp src/DataLoader.cpp -o traffic
./traffic
```

Expected shape of output (your numbers will differ; the timestamps are expected to start at 2012-03-01 00:00:00 and step by 5 minutes):

```
Loaded 34272 rows
Speeds in first row: 207
2012-03-01 00:00:00 | 64.375 67.625 67.125
2012-03-01 00:05:00 | 62.667 68.556 65.444
2012-03-01 00:10:00 | 64.0 67.0 66.0
...
```

Zeros in the speeds may appear. In this dataset a 0 often means a missing reading. Do not fix it tonight; just notice it.

2.4 Debugging table

Symptom	Likely cause	Fix
Error: cannot open ...	Wrong folder or wrong file name	cd to the repo root; check the path with ls
Loaded 0 rows	File empty, or only a header	Open the CSV in a text editor
terminate called ... stod / invalid_argument	A non-numeric cell: text, quotes, or an empty value	Print the field before converting; fix the file; missing-value handling is the next session
Speeds in first row is not 207	Trailing comma, or the first column is not the timestamp	Look at the end of line 2 and the start of line 2
Row count is 34273 instead of 34272	Line 1 is data, not a header	Remove the Block 2 read
undefined reference to loadCSV	DataLoader.cpp missing from the g++ command	Add src/DataLoader.cpp

2.5 Commit

Commit code only. Before committing, run git status. The full CSV and the compiled program (traffic) must not be listed as new files. If they are, add them to .gitignore first:

```
# .gitignore additions
data/raw/
data/traffic_data.csv
traffic

git status
git add .gitignore src/DataLoader.h src/DataLoader.cpp src/main.cpp
git commit -m "Add wide-format CSV loader (TrafficRecord with speeds vector) and test main"
git show --stat
git push
```

git show --stat lists the files in the commit. It should show only the four files above (plus docs/data_notes.md if you commit it separately).

2.6 Self-check: explain aloud before committing

1. What is the difference between a declaration and a definition? What does #pragma once prevent?
2. Why does one CSV line become one TrafficRecord here, and what does speeds[2] refer to?
3. What does std::getline(file, line) do differently from std::getline(ss, field, ',')?
4. Why must you run ./traffic from the repo root?
5. What happens if std::stod receives text that is not a number? Is that handled tonight?
6. Why is the full CSV in .gitignore?
7. The struct has no sensor IDs. Which Task 1 fact tells you which sensor speeds[2] belongs to?

2.7 What not to do tonight

- No ID loading, no sensor selection, no missing-value handling, no timestamp parsing, no optimisation.
- No extra classes or files. Three files only: DataLoader.h, DataLoader.cpp, main.cpp.
- Do not change the struct. If you think it needs changing, write down why and bring it to the next session.

2.8 Task 2 success criteria

- Compiles with no warnings under -Wall -Wextra.
- Prints the row count (lines minus 1), speeds per row (expected 207), and 5 rows of timestamp plus first 3 speeds that match the CSV.
- git show --stat does not contain the big CSV or the binary.
- You answered all seven self-check questions without looking.

Deferred, in this order (not for these tasks)

1. Load graph_sensor_ids.txt so each position maps to an ID in code (uses your Task 1 note).
2. Choose the sensor subset: connected in the distances file, at least one pair with two routes. Required before Dijkstra.
3. Missing values (zeros) and timestamp parsing.

Tell me when you are ready and which topic you are focusing on.